

Full Stack Web Development



CHAPTER-5

Bundled validation



Bundled Validation

Bundled Validation refers to a set of **predefined validators** that are **built into the Struts 2 framework**. These validators are designed to perform common validation tasks such as checking for:

required fields

valid email formats

length of a string

numeric ranges (int/double)

valid dates

URL format

matching patterns using regular expressions

These validators can be used directly **without writing any validation logic manually**, hence the name "**bundled**" (they come *bundled* with Struts 2).



Key Features of Bundled Validation

- 1. Ease of Use:**
Just annotate your fields or configure in `validators.xml`, and Struts will take care of the validation.
- 2. No Custom Code Needed:**
You don't need to write Java code for common checks like "is field required?" or "is age between 18 and 60?".
- 3. Multiple Ways to Apply:**
 - **Annotations** (e.g., `@RequiredStringValidator`)
 - **XML configuration** (`validators.xml`)
You can use either approach or combine both.
- 4. Client-Side Support:**
When using Struts 2 with plugins like Dojo or AJAX, some validators can also provide **client-side validation**.
- 5. Automatic Execution:**
These validators are triggered automatically during the request processing lifecycle **before** action methods are executed.
- 6. Field-Level or Action-Level:**
 - **Field-level validators** apply to specific fields.
 - **Action-level validators** can validate logic involving multiple fields (like confirm password).





Common Scenarios Where Bundled Validation Helps

Validation Need	Bundled Validator
Check if field is not empty	<code>requiredstring</code>
Validate email address	<code>email</code>
Validate username length	<code>stringlength</code>
Ensure age is between 18-60	<code>int (IntRangeFieldValidator)</code>
Validate salary is a double	<code>double (DoubleRangeFieldValidator)</code>
Check date format/range	<code>date or daterange</code>
Confirm URL format	<code>url</code>
Match against regex	<code>regex</code>



1. Required String Validator (requiredstring)

Purpose:

Ensures the string is **not null** and **not empty (after trimming)**. It is commonly used to validate mandatory fields like names, passwords, etc.

Annotation:

```
@RequiredStringValidator(message = "Name is required")
```

```
private String name;
```

XML:-

```
<field name="name">
```

```
  <field-validator type="requiredstring">
```

```
    <message>Name is required</message>
```

```
  </field-validator>
```

```
</field>
```



2. String Length Validator (`stringlength`)

Purpose:

Checks whether the **length of a string** is within a specified range (`minLength` to `maxLength`).

Annotation:

```
@StringLengthValidator(minLength = "5", maxLength = "15", message = "Username must be 5–15 characters")
```

```
private String username;
```

XML:-

```
<field name="username">
```

```
  <field-validator type="stringlength">
```

```
    <param name="minLength">5</param>
```

```
    <param name="maxLength">15</param>
```

```
    <message>Username must be 5–15 characters</message>
```

```
  </field-validator>
```

```
</field>
```



3. Email Validator (email)

Purpose:

Checks whether the input is a **valid email address**.

Annotation:

```
@EmailValidator(message = "Enter a valid email address")
```

```
private String email;
```

XML:-

```
<field name="email">
```

```
    <field-validator type="email">
```

```
        <message>Enter a valid email address</message>
```

```
    </field-validator>
```

```
</field>
```



4. Date Validator (date, daterange)

Purpose:

Validates if the input is a **valid date** and optionally checks if it's within a **given range**.

Annotation:

```
@DateRangeFieldValidator(min = "01/01/2000", max = "12/31/2030", message = "Date must be between 2000 and 2030")
```

```
private Date dob;
```

XML

```
<field name="dob">
```

```
  <field-validator type="daterange">
```

```
    <param name="min">01/01/2000</param>
```

```
    <param name="max">12/31/2030</param>
```

```
    <message>Date must be between 2000 and 2030</message>
```

```
  </field-validator>
```



5. Integer Validator (`int`)

Purpose:

Validates if the input is an **integer** and optionally **within a range**.

Annotation:

```
@IntRangeFieldValidator(min = "18", max = "60", message = "Age must be between 18 and 60")
```

```
private int age;
```

XML:-

```
<field name="age">  
  <field-validator type="int">  
    <param name="min">18</param>  
    <param name="max">60</param>  
    <message>Age must be between 18 and 60</message>  
  </field-validator>  
</field>
```



6. Double Validator (double)

Purpose:

Validates if the input is a **double/decimal** value and optionally **within a range**.

Annotation:

```
@DoubleRangeFieldValidator(minInclusive = "0.0", maxInclusive = "100.0", message = "Marks must be between 0 and 100")
```

```
private double marks;
```

XML:-

```
<field name="marks">
```

```
  <field-validator type="double">
```

```
    <param name="minInclusive">0.0</param>
```

```
    <param name="maxInclusive">100.0</param>
```

```
    <message>Marks must be between 0 and 100</message>
```

```
  </field-validator>
```

```
</field>
```



7. URL Validator (url)

Purpose:

Checks if the input is in **valid URL format** (<http://>, <https://>).

Annotation:

```
@UrlValidator(message = "Enter a valid website URL")
```

```
private String website;
```

XML:-

```
<field name="website">  
  <field-validator type="url">  
    <message>Enter a valid website URL</message>  
  </field-validator>  
</field>
```

8. Regular Expression Validator (**regex**)

Purpose:

Validates the field using a **custom regular expression pattern**.

Annotation:

```
@RegexFieldValidator(regex = "^[A-Z][a-z]{2,15}$", message = "Name must start with a capital letter and be 3-15 letters")
```

```
private String name;
```

XML

```
<field name="name">
```

```
  <field-validator type="regex">
```

```
    <param name="regex">^[A-Z][a-z]{2,15}$</param>
```

```
    <message>Name must start with capital and be 3–15 letters</message>
```

```
  </field-validator>
```

```
</field>
```

9. Custom Validator

Purpose:

Used when **bundled validators are not sufficient** for specific business rules.

Steps:

Step 1: Create a Validator Class

```
public class StartsWithValidator extends FieldValidatorSupport {  
  
    public void validate(Object object) throws ValidationException {  
  
        String value = (String) getFieldValue(getFieldName(), object);  
  
        if (value != null && !value.startsWith("ABC")) {  
  
            addFieldError(getFieldName(), object);  
  
        }  
  
    }  
  
}
```


CONT.

Step 2: Register in validators.xml

```
<validators>  
  <validator name="startswith" class="com.example.StartsWithValidator"/>  
</validators>
```

Step 3: Use in Action XML

```
<field name="customField">  
  <field-validator type="startswith">  
    <message>Field must start with ABC</message>  
  </field-validator>  
</field>
```

Real-life Use:

Used for validations like:

"Username must start with org_"

"PIN must match company format"

"Code must begin with XYZ and be 8 characters long"

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



[www.paruluniversity.ac.](http://www.paruluniversity.ac.in)

[in](http://www.paruluniversity.ac.in)

